



AI/LLM Adoption Tracker

Real-time GitHub Events Processing

Hệ thống xử lý dữ liệu luồng thời gian thực nhằm theo dõi mức độ quan tâm của cộng đồng lập trình viên đối với các công nghệ AI và LLM trên GitHub.

Giảng viên:

TS. Trần Hồng Việt
ThS. Ngô Minh Hương

Sinh viên thực hiện (Nhóm 16):

Lê Thị Khánh Linh - 24022384
Trần Bùi Hà Giang - 24022314

Nội dung trình bày

- Tổng quan dự án & Đặt vấn đề
- Kiến trúc hệ thống & Luồng dữ liệu (Pipeline)
- Chi tiết triển khai các tầng Công nghệ
- Kết quả thực nghiệm & Đánh giá sức bền
- Kết luận & Hướng phát triển

Bối cảnh và Động lực

Vấn đề thực tiễn:

- Trong bối cảnh hiện nay, tần suất tương tác với các công nghệ AI mới (RAG, Claude, AI Agent...) đang thay đổi liên tục theo từng ngày.
- Sự dịch chuyển xu hướng này được phản ánh trực tiếp qua các hành vi của lập trình viên trên nền tảng GitHub.
- Các phương pháp phân tích dữ liệu theo lô (Batch Processing) truyền thống thường tạo ra độ trễ lớn, không bắt kịp những xu hướng công nghệ mang tính thời điểm.

Nhu cầu cấp thiết:

Từ những hạn chế trên, dự án đặt ra yêu cầu xây dựng một hệ thống có khả năng thu thập, xử lý và trực quan hóa dữ liệu luồng với độ trễ tính bằng giây.



Mục tiêu và Phạm vi dự án

Mục tiêu kỹ thuật

- Xây dựng đường ống thu thập sự kiện liên tục từ GitHub API.
- Xử lý luồng dữ liệu thời gian thực bằng Kafka và Spark Streaming.
- Trực quan hóa tần suất từ khóa AI trên Dashboard hiển thị tức thì.

Phạm vi dữ liệu

Xử lý 4 loại sự kiện chính:

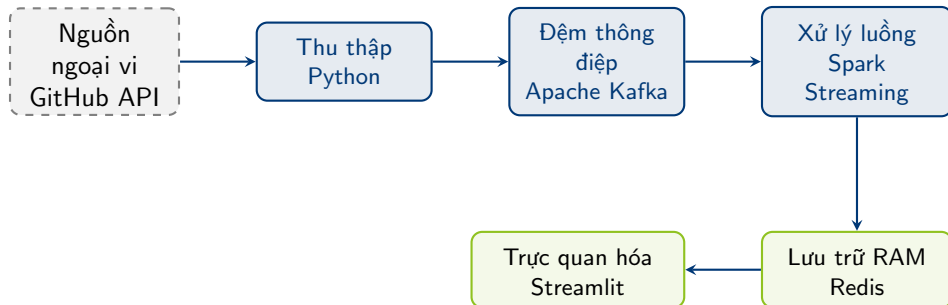
- PushEvent
- PullRequestEvent
- CreateEvent
- WatchEvent

Giới hạn hệ thống

- Cốt lõi dựa trên đối sánh từ khóa.
- Chưa dùng NLP để hiểu ngữ cảnh.
- Chưa triển khai lưu trữ dữ liệu thô dài hạn.

Kiến trúc tổng thể hệ thống

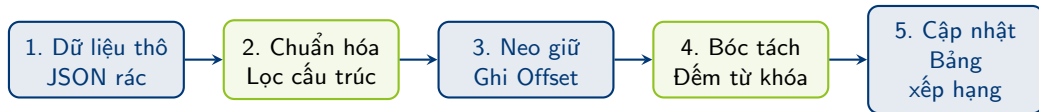
- Hệ thống lõi phân tán bao gồm 5 tầng độc lập, giảm thiểu độ phụ thuộc.
- Dễ dàng mở rộng quy mô và khoanh vùng sự cố để tự phục hồi.



Luồng dữ liệu chi tiết

Đặc tính chuyển động:

- Xử lý liên tục theo mô hình vi lô (Micro-batch) với chu kỳ mỗi 30 giây.
- Đảm bảo tính nhất quán từ khâu thô đến lúc hiển thị biểu đồ.



Bảng tổng hợp Công nghệ

Bộ công nghệ (Tech Stack) đạt tiêu chuẩn công nghiệp, được tối ưu hóa hoàn toàn cho việc xử lý dữ liệu luồng.

Thành phần	Công nghệ	Vai trò trong hệ thống
Nguồn cấp	GitHub API	Cung cấp luồng sự kiện thô liên tục
Thu thập	Python	Lọc, chuẩn hóa JSON và đẩy dữ liệu
Đệm thông điệp	Apache Kafka	Bộ đệm tốc độ cao, quản lý vị trí đọc
Động cơ xử lý	Spark Streaming	Xử lý vi lô, tính toán cửa sổ thời gian
Lưu trữ RAM	Redis	Lưu trữ siêu tốc, phục vụ truy vấn tức thời
Trực quan hóa	Streamlit	Kết xuất giao diện tương tác
Hạ tầng	Docker Compose	Đóng gói và định tuyến mạng phân tán

Tầng thu thập dữ liệu (Ingestion)

Chức năng cốt lõi:

- Gọi API GitHub theo chu kỳ định sẵn.
- Quản lý vòng đời mã xác thực để tối ưu giới hạn gọi hàm.
- Sàng lọc các sự kiện không chứa văn bản ngay từ nguồn.

Biên đổi cấu trúc: Lược bỏ dữ liệu thừa, gom gọn thành định dạng JSON chuẩn bị đẩy vào Kafka.

Cấu trúc JSON xuất ra

```
{  
  "event_id": "12345",  
  "event_type": "PushEvent",  
  "repo_name": "user/rag-demo",  
  "text_content": "Add LangChain pipeline",  
  "created_at": "2026-06-19T10:00Z"  
}
```


Apache Kafka - Trái tim hệ thống

Bài toán giải quyết: Hiện tượng thất cổ chai khi nguồn cấp bùng nổ dữ liệu.

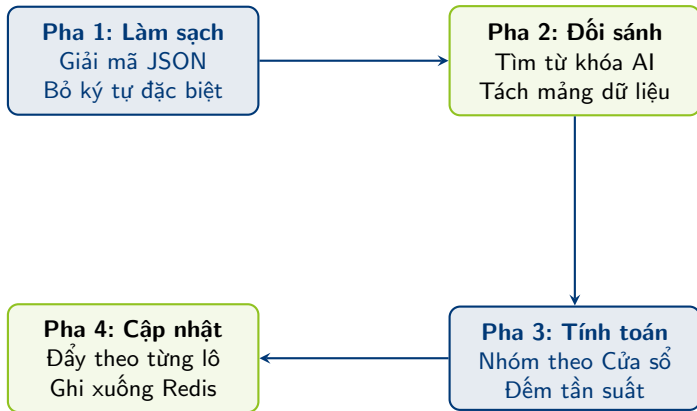
Vai trò chiến lược:

- Tách biệt hoàn toàn nhịp độ thu thập và nhịp độ xử lý tính toán.
- Cung cấp vùng đệm an toàn trên ổ cứng.
- Định vị chính xác dữ liệu đã đọc thông qua cơ chế **Offset**.



Xử lý dữ liệu bằng Spark Streaming

Quy trình xử lý 4 pha bên trong Động cơ Spark:



Thiết kế Lưu trữ với Redis

Lý do lựa chọn: Triệt tiêu độ trễ đọc/ghi ổ cứng, sử dụng cấu trúc dữ liệu trên RAM lý tưởng cho Dashboard thời gian thực.

Cấu trúc Redis	Tên Khóa	Mục đích lưu trữ
ZSET	top_ai_keywords	Bảng xếp hạng từ khóa AI tự động sắp xếp
ZSET	top_ai_repos	Bảng xếp hạng kho lưu trữ sôi động nhất
String	total_ai_events	Bộ đếm cộng dồn tổng sự kiện bất được
List	ai_activity_timeline	Chuỗi thời gian phục vụ vẽ biểu đồ đường

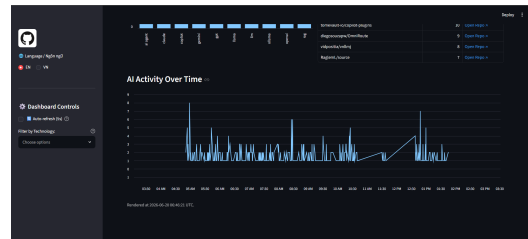
Dashboard Trực quan hóa

Trải nghiệm người dùng:

- Tích hợp đa ngôn ngữ (Việt/Anh).
- Chế độ tự động làm mới kèm đồng hồ đếm ngược.

Tính năng theo dõi:

- Chỉ số tổng quan nhảy số liên tục.
- Bảng xếp hạng định lượng.
- Biểu đồ biến động tương tác theo thời gian.



Giao diện tổng quan và Biểu đồ thời gian thực

Thực nghiệm và Đánh giá tổng quan

Điều kiện vận hành: Chạy tải liên tục ≈ 9 giờ trên cụm Docker.

Kết quả ghi nhận:

- Không xảy ra lỗi tràn RAM (OOM) hay chết tiến trình (Crash) đột ngột.
- Vượt qua các sự cố ngoại cảnh, bảo toàn trọn vẹn luồng dữ liệu lên màn hình hiển thị.

593

Sự kiện AI

0%

Thất thoát dữ liệu

Phân tích Xu hướng Công nghệ

Góc nhìn chuyên sâu từ dữ liệu:

- Kỹ thuật **RAG** dẫn đầu với tần suất áp đảo. Điều này minh chứng cho nhu cầu cấp thiết của cộng đồng trong việc khắc phục điểm yếu của các mô hình ngôn ngữ lớn bằng cách tích hợp dữ liệu nội bộ.
- Sự trỗi dậy của **Claude** (106 lượt) vượt qua các đối thủ sừng sỏ như Gemini (40) hay GPT (27) cho thấy sự chuyển dịch ưu tiên của giới lập trình viên sang các công cụ có khả năng hỗ trợ viết mã ưu việt hơn.
- Làn sóng **AI Agent** đang định hình rõ rệt. Mã nguồn mở đang bước qua thời kỳ “chat” đơn thuần để tiến tới xây dựng các hệ thống tự trị giải quyết vấn đề phức tạp.

Top từ khóa xuất hiện

Công nghệ AI	Số lượt nhắc đến
RAG	156
Claude	106
LLM	90
AI Agent	75
Gemini	40
GPT	27

Sức bền hệ thống

Kiến trúc phân tách dịch vụ (Decoupling) giúp khoanh vùng điểm lỗi, đảm bảo hệ thống tự phục hồi mà không thất thoát dữ liệu.

Kịch bản 1: Gián đoạn kết nối từ nguồn cấp (Lỗi mạng)

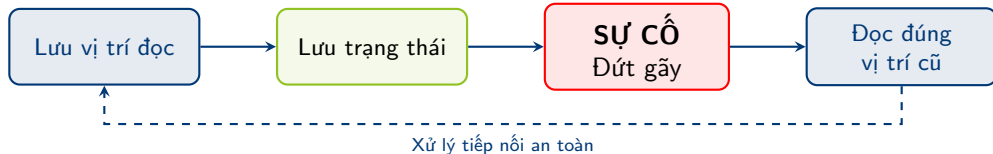
- **Cơ chế:** Tiến trình Python dùng vòng lặp bẫy lỗi để xử lý ngoại lệ *Timeout* từ GitHub API.
- **Hiệu quả:** Cô lập sự cố ở tầng ngoài. Khi có mạng, hệ thống tự động kết nối và tiếp tục đẩy dữ liệu vào Kafka.

Kịch bản 2: Gián đoạn tài nguyên (Máy chủ ngủ đông)

- **Cơ chế:** Luồng tính toán Spark bị đóng băng. Tuy nhiên, Kafka (bộ đệm đĩa cứng) vẫn lưu giữ an toàn các sự kiện mới.
- **Hiệu quả:** Khi thức giấc, Spark truy xuất **Checkpoint** để lấy lại vị trí đã dừng (Offset), tự động xử lý sạch dữ liệu dồn ứ (Backlog) mà không bị trùng lặp.

Cơ chế Tự phục hồi dữ liệu

Chu trình bảo hiểm dữ liệu hoàn hảo:



- Đạt chuẩn cao về khả năng chịu lỗi trong tính toán phân tán.
- Hoàn thiện kiến trúc đường ống luồng dữ liệu khép kín.

Nhận diện Hạn chế

1. Thu thập dữ liệu

Mới khai thác 4 loại sự kiện văn bản, chưa bao phủ toàn diện hệ sinh thái GitHub.

2. Xử lý & Phân tích

Phụ thuộc vào kỹ thuật đối sánh từ khóa, thiếu đánh giá ngữ cảnh nội dung.

3. Lưu trữ dài hạn

Hoàn toàn vắng bóng kho lưu trữ vĩnh viễn, bỏ ngỏ khả năng phân tích xu hướng lịch sử.

Kết luận & Hướng mở rộng

Đã hoàn thành

- Luồng thu thập tự động, tức thời.
- Xử lý vi lỗi ổn định, chịu tải tốt.
- Dashboard báo cáo tương tác.
- Khả năng tự phục hồi ấn tượng sau sự cố.

Hướng phát triển

- Xây dựng Data Lake (HDFS/MongoDB) lưu dữ liệu thô.
- Nhúng mô hình AI (NLP) để phân tích cảm xúc mã nguồn.
- Lập mô hình dự báo xu hướng công nghệ tương lai.

Tài liệu tham khảo

- **Apache Spark** - *Structured Streaming Programming Guide*.
- **Apache Kafka** - *Introduction to Kafka*.
- **Redis** - *Redis Data Types Documentation*.
- **Streamlit** - *Streamlit Library Documentation*.
- **GitHub** - *REST API Events Reference*.

Q & A

Xin trân trọng cảm ơn Thầy/Cô và các bạn đã lắng nghe!



Mã nguồn dự án (GitHub)



Video Demo Hệ thống